

UTN

Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación

Programación 2

Food Store

Sistema de Gestión de Pedidos — TPI

Datos del Estudiante

Joaquin Montero

DNI: 40.007.042 | Matrícula: 102509

Fecha de entrega: 26 de June de 2026

Índice

1.	Marco Teórico	3
1		
.	Java como lenguaje orientado a objetos	3
1		
1		
.	Conceptos clave aplicados	3
2		
1		
.	Herramientas de desarrollo	4
3		
2.	Decisiones Técnicas y Arquitectura	4
2		
.	Organización de paquetes	4
1		
2		
.	Diagrama de relaciones entre clases	5
2		
2		
.	Funcionamiento del sistema	5
3		
3.	Dificultades y Soluciones	6
4.	Bibliografía / Webgrafía	7
5.	Links del Proyecto	7

1. Marco Teórico

1.1 Java como lenguaje orientado a objetos

El proyecto fue desarrollado íntegramente en **Java SE**, un lenguaje de programación orientado a objetos (POO) de alto nivel, fuertemente tipado y de propósito general. Java permite estructurar el código en torno a objetos que encapsulan datos (atributos) y comportamiento (métodos), lo que facilita la reutilización, mantenimiento y escalabilidad de las aplicaciones.

La plataforma **Java SE (Standard Edition)** provee las APIs necesarias para el desarrollo de aplicaciones de consola, incluyendo manejo de colecciones, fechas y horas (*java.time*) y generación de números aleatorios seguros (*java.security.SecureRandom*).

1.2 Conceptos clave aplicados

- **Herencia y clase abstracta:**

La clase abstracta *Base* centraliza los atributos comunes a todas las entidades del sistema: *id* (autoincremental), *eliminado* (borrado lógico) y *createdAt* (timestamp de creación). *Categoria*, *Producto*, *Usuario*, *Pedido* y *DetallePedido* extienden *Base*, heredando estos campos y evitando duplicación de código.

- **Interfaces:**

La interfaz *Calculable* declara el método *calcularTotal()*. La clase *Pedido* la implementa para sumar el subtotal de todos sus detalles, aplicando polimorfismo de comportamiento.

- **Polimorfismo:**

A través de la herencia, todas las entidades pueden ser tratadas como instancias de *Base*, y *Pedido* puede ser tratado como *Calculable*, permitiendo escribir código genérico que opera sobre distintos tipos en tiempo de ejecución.

- **Enumeraciones (Enum):**

Se utilizan tres enums para representar valores discretos de dominio: *Estado* (*PENDIENTE*, *CONFIRMADO*, *TERMINADO*, *CANCELADO*), *FormaPago* (*TARJETA*, *TRANSFERENCIA*, *EFFECTIVO*) y *Rol* (*ADMIN*, *USUARIO*). Los enums garantizan seguridad de tipos y evitan el uso de constantes mágicas.

- **Excepciones personalizadas:**

CadenaVacíaException y *NumeroNegativoException* extienden *RuntimeException*. Son lanzadas desde la capa de UI cuando el usuario ingresa valores inválidos, brindando mensajes de error claros y específicos en lugar de mensajes genéricos del sistema.

- **Colecciones (ArrayList):**

Cada servicio mantiene una lista en memoria con *ArrayList*, que actúa como repositorio temporal de datos durante la ejecución del programa.

- **Borrado lógico:**

En lugar de eliminar objetos de la lista, se activa el flag eliminado = true heredado de Base. Los métodos de listado filtran los registros marcados, preservando la integridad referencial sin necesidad de una base de datos.

- **Generación de contraseña segura:**

La clase Usuario genera automáticamente una contraseña alfanumérica de 8 caracteres usando SecureRandom, garantizando aleatoriedad criptográficamente segura.

1.3 Herramientas de desarrollo

Apache NetBeans IDE 23	Entorno de desarrollo integrado utilizado para escribir, compilar y ejecutar el proyecto Java.
Apache Maven	Sistema de gestión de dependencias y construcción del proyecto, configurado a través del archivo pom.xml.
Java SE 21	Versión de la plataforma utilizada, con soporte LTS y características modernas como switch expressions.
Git / GitHub	Sistema de control de versiones para gestionar el historial del código y mantener el repositorio en línea.

2. Decisiones Técnicas y Arquitectura

2.1 Organización de paquetes

El proyecto adopta una **arquitectura en capas** que separa responsabilidades claramente, facilitando el mantenimiento y la extensibilidad del sistema. Cada paquete agrupa clases con un único propósito:

Paquete	Contenido	Responsabilidad
integrado.prog2	Main.java	Punto de entrada de la aplicación. Inicializa el ciclo principal y delega en
entities	Base, Usuario, Producto, Categoria, Pedido, DetallePedido, Calculable	Entidades de dominio. Define la estructura de datos y sus relaciones.
enums	Estado, FormaPago, Rol	Valores constantes y tipados para estados de dominio.
exception	CadenaVacíaException, NumeroNegativoException	Excepciones personalizadas para validación de entrada de usuario.
service	UsuarioService, ProductoService, CategoriaService, PedidoService	Servicios de colecciones en memoria (repositorio).
ui	MenuPrincipal, MenuUsuarios, MenuProductos, MenuCategorias, MenuPedidos	Interacción con la consola. Captura entrada y delega en

2.2 Diagrama de relaciones entre clases

El siguiente diagrama simplificado muestra las relaciones de herencia, implementación y asociación entre las clases principales del sistema:



2.3 Funcionamiento del sistema

Food Store es una aplicación de consola que implementa un **sistema de gestión de pedidos** para una tienda de comida. Al ejecutarse, presenta un menú principal con cuatro módulos funcionales:

- **Categorías:**
Permite crear, listar, editar y eliminar categorías de productos. Cada categoría tiene nombre, descripción y fecha de creación automática.
- **Productos:**
Gestión completa de productos con nombre, precio, stock, imagen, disponibilidad y categoría asociada. Valida precios y stocks mayores a cero mediante excepciones personalizadas.

- **Usuarios:**

Alta de usuarios con generación automática de contraseña alfanumérica de 8 caracteres (usando `SecureRandom`). Permite actualizar datos de contacto y eliminar usuarios lógicamente.

- **Pedidos:**

Flujo completo de creación: selección de usuario, agregado iterativo de productos con validación de stock disponible, cálculo automático del total mediante la interfaz `Calculable`, y gestión de estado (PENDIENTE → CONFIRMADO → TERMINADO / CANCELADO) y forma de pago.

El flujo de navegación parte de **Main.java**, que instancia `MenuPrincipal`. Este último crea una instancia de cada servicio y cada submenú, inyectando los servicios necesarios como dependencias. El bucle principal captura la opción del usuario y delega la ejecución al submenú correspondiente. Cada submenú gestiona su propio bucle interno y vuelve al menú principal al seleccionar la opción 0.

Validaciones implementadas en la capa UI

- Nombre de producto no puede ser nulo ni estar vacío (verificación con `isBlank()`).
- Precio y stock deben ser mayores a cero — se lanza `NumeroNegativoException` si no se cumple.
- Al seleccionar categoría para un producto, se verifica que exista y no esté eliminada.
- Al crear un pedido, se verifica que el usuario exista y no esté eliminado.
- La cantidad a pedir no puede superar el stock disponible del producto.
- La entrada de opciones numéricas es protegida con bloques try-catch para `NumberFormatException`.

3. Dificultades y Soluciones

A lo largo del desarrollo surgieron distintos obstáculos técnicos. A continuación se describen los principales y cómo fueron resueltos:

1. Diseño de la jerarquía de herencia

Problema: Al comenzar a modelar las entidades, surgió la pregunta de qué atributos compartían todas y cuáles eran específicos de cada clase. Poner id y createdAt en cada clase generaba duplicación de código.

Solución: Se creó la clase abstracta Base con los atributos comunes. Utilizar un contador estático por clase (cantidadCategorias, cantidadProductos, etc.) permitió generar IDs autoincrementales sin base de datos, asignando $id = contador + 1$ en el constructor y luego incrementando el contador.

2. Implementación del borrado lógico

Problema: Eliminar objetos directamente de las listas podría generar errores de referencia si otros objetos los tenían asociados (por ejemplo, un Producto ya referenciado en un DetallePedido).

Solución: Se implementó el patrón de borrado lógico: el atributo eliminado (heredado de Base) se activa con setEliminado(true). Todos los métodos de listado y búsqueda filtran los registros con isEliminado() == true, manteniendo la integridad referencial sin necesidad de manejo complejo de dependencias.

3. Cálculo del total del pedido

Problema: Determinar dónde y cuándo calcular el total de un Pedido fue una decisión de diseño relevante. Calcularlo en el constructor sería prematuro (los detalles se agregan después).

Solución: Se definió la interfaz Calculable con el método calcularTotal(). Pedido la implementa iterando su lista de DetallePedido. El método es llamado explícitamente desde MenuPedidos una vez que el usuario termina de agregar productos, garantizando que el total refleja todos los ítems.

4. Validación de entradas del usuario

Problema: El uso de Integer.parseInt() y Double.parseDouble() lanza NumberFormatException si el usuario ingresa texto no numérico, y no existía un mecanismo claro para comunicar errores de negocio (precio negativo, stock cero).

Solución: Se combinaron dos estrategias: (a) bloques try-catch para NumberFormatException en todas las lecturas numéricas, y (b) excepciones de negocio personalizadas (NumeroNegativoException, CadenaVacíaException) lanzadas cuando el valor numérico es válido pero infringe una regla de negocio. Esto separa los errores de parseo de los errores de validación.

5. Manejo del stock al agregar detalles al pedido

Problema: Al agregar un producto a un pedido, era necesario decrementar el stock disponible para evitar vender más unidades de las existentes, y validar previamente que hubiera suficiente stock.

Solución: En `PedidoService.agregarDetalle()`, antes de agregar el `DetallePedido`, se verifica que `cantidad <= producto.getStock()`. Si pasa la validación, se calcula el subtotal y se descuenta el stock con `producto.setStock(stock - cantidad)`. Esta lógica centralizada en el servicio evita que la capa UI maneje esa responsabilidad.

6. Importación y configuración en NetBeans

Problema: Al importar el proyecto Maven en Apache NetBeans, el IDE no reconocía inicialmente la estructura de paquetes y mostraba errores de compilación.

Solución: Se verificó que el archivo `pom.xml` estuviera correctamente configurado con el `groupId` (integrado.prog2), el `artifactId` y la versión de Java. Usando la opción 'Open Project' de NetBeans (en lugar de 'Import') y esperando a que Maven descargara las dependencias, el proyecto compiló y ejecutó correctamente.

4. Bibliografía / Webgrafía

Los siguientes recursos fueron consultados como referencia durante el desarrollo del proyecto:

[1]	Oracle Corporation — Java SE 21 Documentation — The Java™ Tutorials https://docs.oracle.com/en/java/javase/21/
[2]	Oracle Corporation — Java Language Specification — Classes, Interfaces, Enums https://docs.oracle.com/javase/specs/
[3]	Oracle Corporation — java.time — Date and Time API https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/time/package-summary.html
[4]	Oracle Corporation — java.security.SecureRandom https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/security/SecureRandom.html
[5]	Apache NetBeans — NetBeans IDE Documentation https://netbeans.apache.org/front/main/
[6]	Apache Maven — Maven Getting Started Guide https://maven.apache.org/guides/getting-started/
[7]	Baeldung — Guide to Java Enums / Custom Exceptions / Collections https://www.baeldung.com/java-enum-guide
[8]	GitHub — Repositorio del proyecto — JoaquinMontero96/tpi-programacion2 https://github.com/JoaquinMontero96/tpi-programacion2

5. Links del Proyecto

Recurso	Enlace
Repositorio GitHub	https://github.com/JoaquinMontero96/tpi-programacion2
Video explicativo	[Pendiente de entrega – agregar URL aquí]

Nota: el link al video explicativo debe completarse antes de la entrega final.